CS-GY 6923: Lecture 14
Diffusion and Transformers

NYU Tandon School of Engineering, Akbar Rafiey

News

- Final Dec 19. Same time and location.
- Homework 5 due on Dec 10.

Diffusion Model

Diffusion

Auto-encoder/VAE, GAN approach: Input noise, map directly to image and vice versa.

Diffusion: Slowly move from noise to image and vice versa.

Denoising Diffusion Probabilistic Models

Jonathan Ho
UC Berkeley
jonathanho@berkeley.edu

Ajay Jain
UC Berkeley
ajayj@berkeley.edu

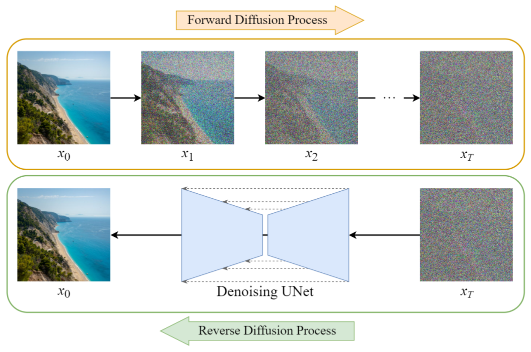
Pieter Abbeel
UC Berkeley
pabbeel@cs.berkeley.edu

Abstract

We present high quality image synthesis results using diffusion probabilistic models, a class of latent variable models inspired by considerations from nonequilibrium thermodynamics. Our best results are obtained by training on a weighted variational bound designed according to a novel connection between diffusion probabilistic

How diffusion models work

- Forward Process:
  - Gradually add noise to data until it becomes pure noise.
- Reverse Process:
  - Train a neural network to remove the noise step by step.

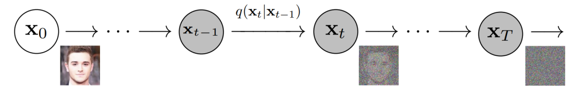


Key Question: How do we predict and reverse noise effectively?

Mathematical formulation

Forward Process (Adding Noise):

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$



- β_t: Noise schedule.
- After T steps, for large enough T, $\mathbf{x}_T$ is pure noise.

Cumulative Noise:

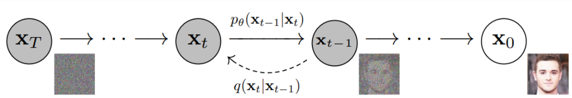
$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

with retention factor $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$.

Mathematical formulation

Reverse Process (Denoising):

$$p_\theta(\mathbf{x}_{t-1}|x_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t))$$



- μ_θ: Predicted mean of the clean image.
- Σ_θ: Predicted variance (optional).

Training objective:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0, t, \epsilon}\left[\|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2\right]$$

Diffusion models vs VAEs

- Recall the goal of VAE was to have a probabilistic representation of latent space attributes. This was done in one shot, from image to Gaussian distributions.
- VAE decoder does the reverse in one shot. Takes Gaussian noise and returns an image.
- Diffusion model doing more or less the same but with many careful intermediate noising and denoising steps.
- There are fundamental differences ...

Diffusion process

- A diffusion process is a stochastic Markov process having continuous path
- stochastic Markov process: future state will only depend on the current state, knowing the past does not change anything
- continuous: no jumps
- Allows to transition from complex distributions to simple distributions

Forward process: a closer look

Forward process: $q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t \mathbf{I})$

Why does it converge to a simple distribution ?

Suppose the process is $\mathbf{x}_t = \alpha \mathbf{x}_{t-1} + \beta \mathcal{N}(0, I)$.

What values of α and β makes sense for the process?

- $\alpha = 0, \beta = 1$?
- $\alpha \geq 1, \beta \leq 1$?
- Seems something like $\alpha = \sqrt{0.99}, \beta = \sqrt{0.01}$ is appropriate.

Forward process: a closer look

Let's check if the following α and β converges:

$$\mathbf{x}_t = \sqrt{1-\beta}\mathbf{x}_{t-1} + \sqrt{\beta}\mathcal{N}(0, I)$$

$$\begin{aligned}
\mathbf{x}_t &= \sqrt{1-\beta}\mathbf{x}_{t-1} + \sqrt{\beta}\mathcal{N}(0, I) \\
&= \sqrt{1-\beta}(\sqrt{1-\beta}\mathbf{x}_{t-2} + \sqrt{\beta}\mathcal{N}(0, I)) + \sqrt{\beta}\mathcal{N}(0, I) \\
&= (\sqrt{1-\beta})^2\mathbf{x}_{t-2} + \sqrt{1-\beta}\sqrt{\beta}\mathcal{N}(0, I) + \sqrt{\beta}\mathcal{N}(0, I)
\end{aligned}$$

Forward process: a closer look

Let's check if the following α and β converges:

$$\mathbf{x}_t = \sqrt{1-\beta}\mathbf{x}_{t-1} + \sqrt{\beta}\mathcal{N}(0, I)$$

$$\begin{aligned}
\mathbf{x}_t &= \sqrt{1-\beta}\mathbf{x}_{t-1} + \sqrt{\beta}\mathcal{N}(0, I) \\
&= \sqrt{1-\beta}(\sqrt{1-\beta}\mathbf{x}_{t-2} + \sqrt{\beta}\mathcal{N}(0, I)) + \sqrt{\beta}\mathcal{N}(0, I) \\
&= (\sqrt{1-\beta})^2\mathbf{x}_{t-2} + \sqrt{1-\beta}\sqrt{\beta}\mathcal{N}(0, I) + \sqrt{\beta}\mathcal{N}(0, I) \\
&\dots \\
&= (\sqrt{1-\beta})^t\mathbf{x}_{t-t} + \cdots + (\sqrt{1-\beta})^2\sqrt{\beta}\mathcal{N}(0, I) + \sqrt{1-\beta}\sqrt{\beta}\mathcal{N}(0, I) \\
&+ \sqrt{\beta}\mathcal{N}(0, I)
\end{aligned}$$

Forward process: a closer look

$$\mathbf{x}_t = (\sqrt{1-\beta})^t \mathbf{x}_0 + \cdots + (\sqrt{1-\beta})^2 \sqrt{\beta}\mathcal{N}(0, I) + \sqrt{1-\beta}\sqrt{\beta}\mathcal{N}(0, I)$$
$$+ \sqrt{\beta}\mathcal{N}(0, I)$$

- for large t, $(\sqrt{1-\beta})^t$ approaches to 0
- each of $(\sqrt{1-\beta})^i \sqrt{\beta}\mathcal{N}(0, I)$ is a Gaussian with mean 0 and variance $(1-\beta)^i \beta$.
- All these Gaussian can be written as one Gaussian distribution with variance:
$$\sum_{i=0}^{t}(1-\beta)^i \beta = \beta \frac{1-(1-\beta)^t}{1-(1-\beta)} \sim \frac{\beta}{\beta} = 1$$

For large enough t we converge to the Normal distribution $\mathcal{N}(0, I)$.

Forward process: a closer look

- In practice we do not use a fixed noise β
- Linear schedule noise: $\beta_1 = 0.0001$ and $\beta_T = 0.02$
- The number of steps is about 10000 (application dependent), but this is slow!
- Recall $\mathbf{x}_t = \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\mathcal{N}(0, I)$.
- Let's define $\alpha_t = 1 - \beta_t$ and do the recursive expansion.

Forward process: a closer look

$$
\begin{aligned}
\mathbf{x}_t &= \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\mathcal{N}(0, I) \\
&= \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{1-\alpha_t}\mathcal{N}(0, I) \\
&= \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1-\alpha_{t-1}}\mathcal{N}(0, I)) + \sqrt{1-\alpha_t}\mathcal{N}(0, I) \\
&= (\sqrt{\alpha_t\alpha_{t-1}})\mathbf{x}_{t-2} + \sqrt{1-\alpha_t+\alpha_t-\alpha_t\alpha_{t-1}}\sqrt{\beta}\mathcal{N}(0, I) \\
&\ldots
\end{aligned}
$$

Forward process: a closer look

$$
\begin{aligned}
\mathbf{x}_t &= \sqrt{1-\beta_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\mathcal{N}(0, I) \\
&= \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{1-\alpha_t}\mathcal{N}(0, I) \\
&= \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1-\alpha_{t-1}}\mathcal{N}(0, I)) + \sqrt{1-\alpha_t}\mathcal{N}(0, I) \\
&= (\sqrt{\alpha_t\alpha_{t-1}})\mathbf{x}_{t-2} + \sqrt{1-\alpha_t+\alpha_t-\alpha_t\alpha_{t-1}}\mathcal{N}(0, I) \\
& \dots \\
&= \sqrt{\alpha_t\alpha_{t-1}\cdots\alpha_2\alpha_1}\mathbf{x}_0 + \sqrt{1-\alpha_t\alpha_{t-1}\cdots\alpha_2\alpha_1}\mathcal{N}(0, I) \\
&= \sqrt{\prod_{i=1}^{t}\alpha_i}\mathbf{x}_0 + \sqrt{1-\prod_{i=1}^{t}\alpha_i}\mathcal{N}(0, I) \\
&= \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\mathcal{N}(0, I) \\
&= \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon
\end{aligned}
$$

Reverse process: a closer look

Computing the reverse path (reverse denoising distribution) is not easy, but we know it is diffusion process.

We train a model to approximate the reverse distribution.

Similar to VAEs, the goal is to maximize a lower bound for the likelihood:

$$\log p(\mathbf{x}_0) \geq \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\left[\log \frac{p(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)}\right]$$

A lot of effort goes into simplify this lower bound and exploiting the fact that we are dealing with diffusion process.

Reverse process: a closer look

At the end of the day, the model should learn the noise added

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0, t, \epsilon}\left[\|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2\right]$$

What does it mean? How do we actually do training ?

Reverse process: training

- Sample an image $\mathbf{x}_0$ and a time step t
- Sample a random noise ε.
- Get the noise image at time t: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon$
- Feed $\mathbf{x}_t$ to the neural network.
- The model's predicted noise ε_θ should be close to ε.

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Reverse process: image generation

- Start from a noise image $\mathbf{x}_T \sim \mathcal{N}(0, I)$.
- Feed this to the trained neural network to predict a noise $\varepsilon_\theta(\mathbf{x}_T)$
- Given this predicted noise we can do denoising and obtain $\mathbf{x}_{T-1}$ (we skipped through the math here)
- Repeat the above until we get to $\mathbf{x}_0$.

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\quad \mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Semantic embeddings + diffusion

Text to image synthesis: Dall-E, Imagen, Stable Diffusion

"A chair that looks like a pineapple"

Transformers

Text processing

We have seen semantic embeddings for words.

- An AI model.
- A fashion model.

The restaurant refused to serve me a ham sandwich because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambiance was just as good as the food and service.

- How to process this text into representation for down stream tasks ?
  - Positive or negative review ?
  - "Does the restaurant serve steak?"
- What are some challenges we face?

Text processing

What are some challenges we face?

- Input can be large: 37 words each with 1024 dimensional representation gives 37*1034=37888. Fully connected NN is not feasible for long text.
- Different length text, hence different length representation.

Observation: The network should share parameters across different words in different positions; similar to convolution.

We need a model where parameters don't increase with input length.

Text processing

What are some challenges we face?

The restaurant refused to serve me a ham sandwich because it only cooks vegetarian food. In the end, they just gave me two slices of bread. Their ambiance was just as good as the food and service.

- The word *their* must **attend to** the word *restaurant*.

Observation: There must be connections between the words.

The strength of these connections will depend on the words themselves.

The connections should span across the text.

Dot product self attention

Dot product self attention

Transformer acquires these properties using dot product self attention.

- Takes N inputs $\mathbf{x}_1, \ldots, \mathbf{x}_N$ of size $D \times 1$ and returns N outputs of size $D \times 1$
- First, computes N values (no ReLU), one value for each input

$$\mathbf{v}_i = \mathbf{b}_v + \mathbf{W}_v\ \mathbf{x}_i \qquad \text{with}\ \ \mathbf{b}_v \in \mathbb{R}^D,\ \ \mathbf{W}_v \in \mathbb{R}^{D \times D}$$

Dot product self attention

Transformer acquires these properties using dot product self attention.

- Takes N inputs $\mathbf{x}_1, \ldots, \mathbf{x}_N$ of size $D \times 1$ and returns N outputs of size $D \times 1$
- First, computes N values (no ReLU), one value for each input

$$\mathbf{v}_i = \mathbf{b}_v + \mathbf{W}_v\ \mathbf{x}_i \qquad \text{with}\ \ \mathbf{b}_v \in \mathbb{R}^D,\ \ \mathbf{W} \in \mathbb{R}^{D \times D}$$

- The i-th output, $\mathbf{sa}_i[\mathbf{x}_1, \ldots, \mathbf{x}_N]$ is a *weighted sum* of over all values $\mathbf{v}_1, \ldots, \mathbf{v}_N$.

$$\mathbf{sa}_i[\mathbf{x}_1, \ldots, \mathbf{x}_N] = \sum_{m=1}^{N} a[\mathbf{x}_m, \mathbf{x}_i] \cdot \mathbf{v}_m$$

$a[\mathbf{x}_m, \mathbf{x}_i]$ is the *attention* the i-th output pays to input m.

Dot product self attention

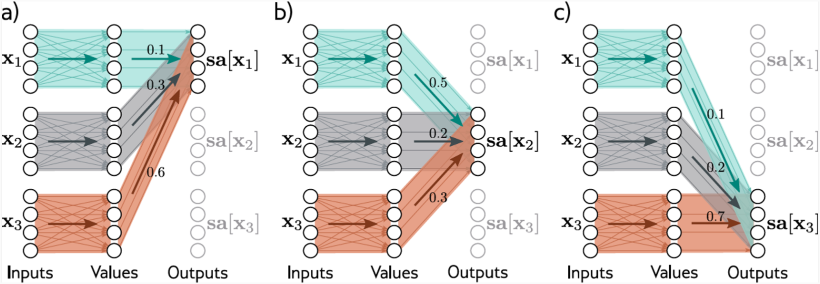


Image from *Understanding Deep Learning* by Simon Prince

Attention weights

- Attention weights are non-linear functions of input.
- Compute N queries and N keys from input

$$\mathbf{q}_i = \mathbf{b}_q + \mathbf{W}_q \mathbf{x}_i$$
$$\mathbf{k}_i = \mathbf{b}_k + \mathbf{W}_k \mathbf{x}_i$$

- Compute similarities and pass through softmax:

$$a[\mathbf{x}_m, \mathbf{x}_i] = \text{softmax}_m(\text{sim}(\mathbf{k}_m, \mathbf{q}_i))$$

We can use vector dot product as a measure of similarity.

Attention weights

- Attention weights are non-linear functions of input.
- Compute N queries and N keys from input

$$\mathbf{q}_i = \mathbf{b}_q + \mathbf{W}_q \mathbf{x}_i$$
$$\mathbf{k}_i = \mathbf{b}_k + \mathbf{W}_k \mathbf{x}_i$$

- Compute similarities and pass through softmax:

$$\begin{aligned} a[\mathbf{x}_m, \mathbf{x}_i] &= \mathrm{softmax}_m(\mathbf{k}_m^\top \mathbf{q}_i) \\ &= \frac{\exp\left(\mathbf{k}_m^\top \mathbf{q}_i\right)}{\sum_{j=1}^{N} \exp\left(\mathbf{k}_j^\top \mathbf{q}_i\right)} \end{aligned}$$

Attention weights

- For each input $\mathbf{x}_i$ the attention weights are positive and sum to one.
- The attention weights depend on the relative similarities between the i-th query and all of the keys.
- The queries and keys must have the same dimensions. However, these can differ from the dimension of the values, which is usually the same size as the input.
- Note there is no activation function, but the mechanism is nonlinear due to the dot-product and a softmax operation used to compute the attention weights.

Attention weights

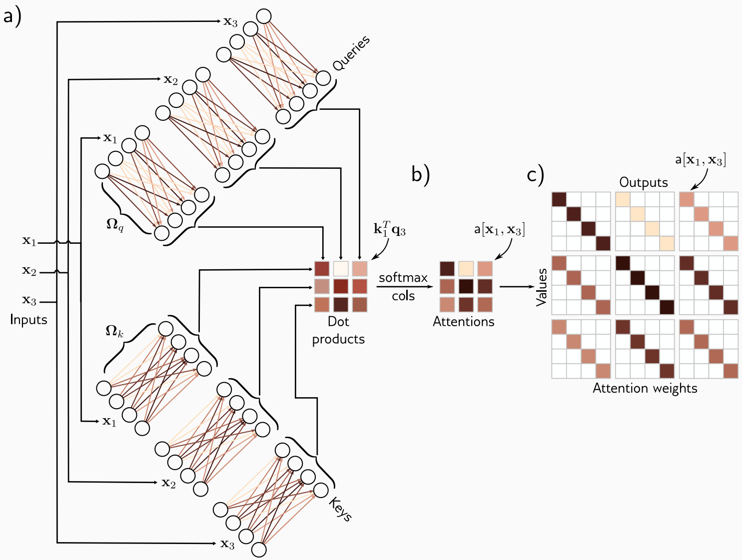


Image from *Understanding Deep Learning* by Simon Prince

What do we have so far?

- First, there is a single shared set of parameters $\phi = \{\mathbf{b}_v, \mathbf{W}_v, \mathbf{b}_q, \mathbf{W}_q, \mathbf{b}_k, \mathbf{W}_k\}$. This is independent of the number of inputs N, so the network can be applied to different sequence lengths.
- Second, there are connections between the inputs (words), and the strength of these connections depends on the inputs themselves via the attention weights.

Matrix form

- Data matrix $\mathbf{X}$
- $\mathbf{V} = \mathbf{b}_v \mathbf{1}^\top + \mathbf{W}_v \mathbf{X} \quad (\mathbf{1} \text{ is } N \times 1\ )$
- $\mathbf{Q} = \mathbf{b}_q \mathbf{1}^\top + \mathbf{W}_q \mathbf{X}$
- $\mathbf{K} = \mathbf{b}_k \mathbf{1}^\top + \mathbf{W}_k \mathbf{X}$
- $\mathbf{Sa}[\mathbf{X}] = \mathbf{V} \cdot \mathrm{softmax}(\mathbf{K}^\top \mathbf{Q})$ performs the softmax operation independently on each of its columns
- Often we use the scaled version $\mathbf{Sa}[\mathbf{X}] = \mathbf{V} \cdot \mathrm{softmax}\left(\frac{\mathbf{K}^\top \mathbf{Q}}{\sqrt{D_q}}\right)$. Scaled attention converges faster and yields better results.

Matrix form

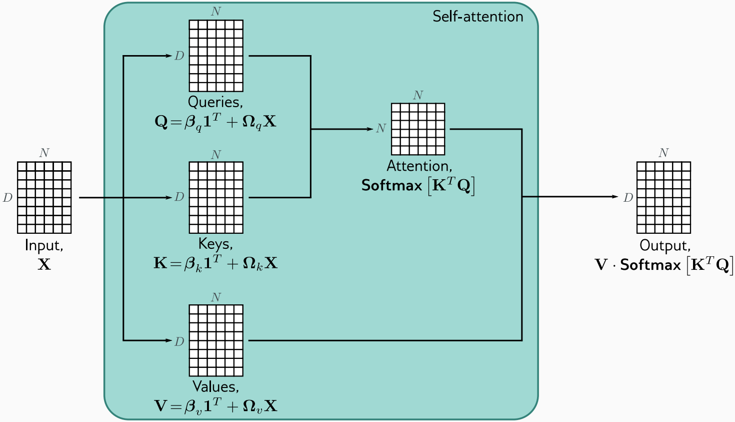


Image from *Understanding Deep Learning* by Simon Prince

Positional encoding

So far we have ignored the order of input and our set up is permutation equivariant.

The sentence The woman ate the raccoon has a different meaning than The raccoon ate the woman.

Two main approaches:

- Absolute positional encodings
- Relative positional encodings

$$\mathbf{Sa}[\mathbf{X}] = (\mathbf{V} + \mathbf{\Pi}) \cdot \mathrm{softmax}((\mathbf{K}^\top + \mathbf{\Pi})(\mathbf{Q} + \mathbf{\Pi}))$$

Multiple heads

Multiple self-attention mechanisms are usually applied in parallel, and this is known as multi-head self-attention. Now H different sets of values, keys, and queries are computed:

- Data matrix $\mathbf{X}$
- $\mathbf{V}_h = \mathbf{b}_{vh}\mathbf{1}^\top + \mathbf{W}_{vh}\mathbf{X} \quad$ ($\mathbf{1}$ is $N \times 1$)
- $\mathbf{Q}_h = \mathbf{b}_{qh}\mathbf{1}^\top + \mathbf{W}_{qh}\mathbf{X}$
- $\mathbf{K}_h = \mathbf{b}_{kh}\mathbf{1}^\top + \mathbf{W}_{kh}\mathbf{X}$
- The h-th self-attention mechanism or head can be written as: $\mathbf{Sa}_h[\mathbf{X}] = \mathbf{V}_h \cdot \mathrm{softmax}(\mathbf{K}_h^\top \mathbf{Q}_h)$

Multiple heads

Typically, if the dimension of the inputs $\mathbf{x}_i$ is D and there are H heads, the values, queries, and keys will all be of size D/H, as this allows for an efficient implementation. The outputs of these self-attention mechanisms are vertically concatenated, and another linear transform $\mathbf{W}_c$ is applied to combine them.

$$\mathbf{MhSa}[\mathbf{X}] = \mathbf{W}_c[\mathbf{Sa}_1[\mathbf{X}]^\top, \mathbf{Sa}_2[\mathbf{X}]^\top, \ldots, \mathbf{Sa}_H[\mathbf{X}]^\top]^\top$$

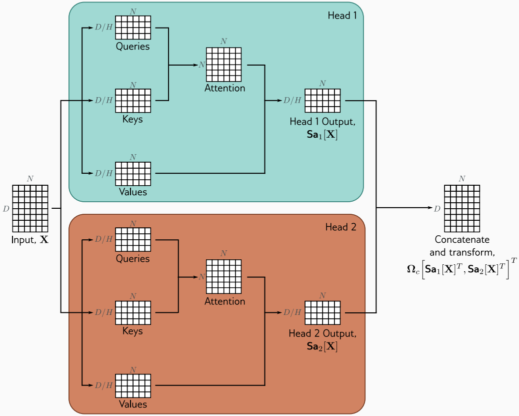


Image: *Understanding Deep Learning* by S. Prince

Transformer layers

In a real network, the data passes through a series of these transformer layers.

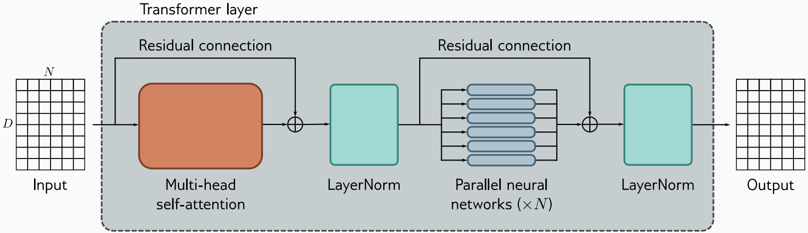


Image: *Understanding Deep Learning* by S. Prince

Transformers for natural language processing

A generic pipeline in NLP that uses transformers is as follows:

- **Tokenization**: split the text into common words and word fragments from which larger and less frequent words can be composed. Typical vocabulary size: 30,000
- **Embedding**: applies embedding to a bag-of-word matrix to get embedding for each word/token (previous lecture). Typical embedding dimension = 1024
- **Embedding** the embedding matrix goes through K transformers layer.
  - Encoder: transforms the text embeddings into a representation that can support a variety of tasks. Ex: BERT
  - Decoder: predicts the next token to continue the input text. Ex:GPT

Encoder model example: BERT

Architecture: A 24-layer Transformer with 16-head self-attention (each head uses 64-dim queries/keys/values. the matrices $\mathbf{W}_{vh}, \mathbf{W}_{qh}, \mathbf{W}_{kh}$ are 64×1024). The MLP size in each layer is 4096. Total of 340M parameters.

BERT pre-training

- Pre-training uses self-supervision, enabling training on massive text corpus without manual labels.
- BERT's self-supervised task: predicting masked (missing) words from internet text.
- Training setup:
    - Maximum input length: 512 tokens
    - Batch size: 256
    - 1 million training steps (about 50 epochs over a 3.3-billion-word corpus)
- Predicting missing words encourages the model to learn:
    - Syntax (e.g., "red" appears before nouns, not verbs)
    - Basic commonsense patterns (e.g., "train" is more likely than "peanut" in "The <mask> pulled into the station")

Fine tuning

Model parameters are adjusted to specialize the pretrained network for a specific task. An additional output layer is added to map transformer outputs to the required prediction format.

Examples of fine-tuning tasks:

- **Text classification (e.g., sentiment analysis):**
  - A special `<CLS>` token is prepended during pre-training.
  - Its final hidden vector is mapped to a single value and passed through a sigmoid.
  - Trained using binary cross-entropy loss.
- **Word classification (e.g., named entity recognition):**
  - Each token embedding x_n is mapped to an $E \times 1$ vector (one score per entity type).
  - A softmax over the E classes gives per-token class probabilities.
  - Trained using multiclass cross-entropy loss.

Decoder model example: GPT3

- GPT-3 is an example of a **decoder-only** transformer model.
- Its architecture is similar to the encoder: stacked Transformer layers applied to learned word embeddings.
- Difference in purpose:
  - Encoder models: build text representations for downstream NLP tasks.
  - Decoder models: generate the **next token** in a sequence.
- By repeatedly feeding its own output back as input, the decoder can generate coherent text passages.

Decoder model example: GPT3

GPT-3 is an autoregressive language model: it predicts each token based on all previous tokens.

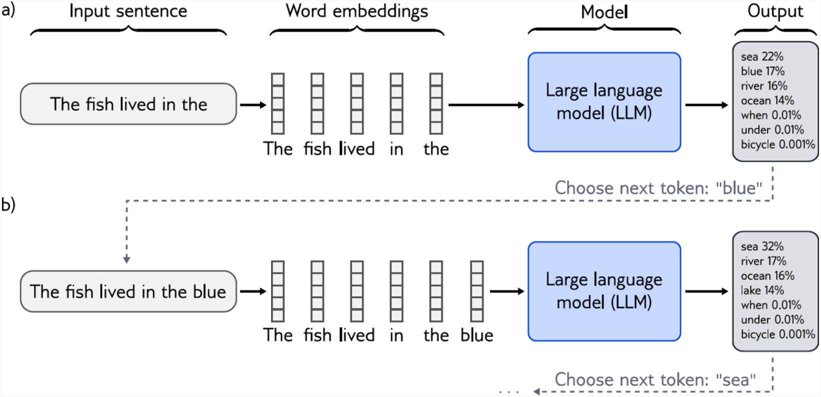


Image by: S. Prince

Language modeling

The probability of the entire sentence is factored into conditional probabilities:

$$
\begin{aligned}
Pr(\text{It takes great courage to let yourself appear weak}) \quad = \\
& Pr(\text{It}) \times Pr(\text{takes}|\text{It}) \times Pr(\text{great}|\text{It takes}) \times Pr(\text{courage}|\text{It takes great}) \times \\
& Pr(\text{to}|\text{It takes great courage}) \times Pr(\text{let}|\text{It takes great courage to}) \times \\
& Pr(\text{yourself}|\text{It takes great courage to let}) \times \\
& Pr(\text{appear}|\text{It takes great courage to let yourself}) \times \\
& Pr(\text{weak}|\text{It takes great courage to let yourself appear}). \qquad (12.14)
\end{aligned}
$$

Example: *Understanding Deep Learning* by S. Prince

By multiplying these conditional probabilities, the model implicitly defines the **joint probability** of the full token sequence:

$$Pr(t_1, t_2, \ldots, t_N) = \prod_{n=1}^{N} Pr(t_n \mid t_1, \ldots, t_{n-1}).$$

Next-token prediction and joint probability modeling are two views of the same underlying task.

Predicting all next words simultaneously?

A decoder is trained by maximizing the log-probability of the next token at every position. How can we prevent “cheating” i.e. blocking access to future words?

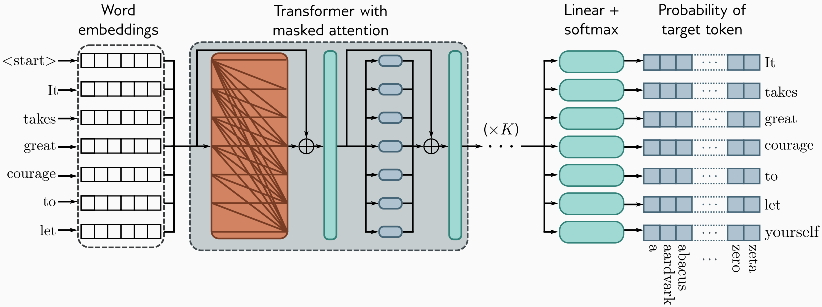
How can we predict all the next words simultaneously using attention mechanism ?

Masked attention

Fast and scalable training that maintains the autoregressive constraints required for language modeling.

- **Masked self-attention prevents cheating:** Each token can only attend to itself and earlier tokens, blocking access to future words.
- **Enables parallel next-token prediction:** With the mask in place, the model can predict the next word at every position in the sequence in a single forward pass.
- **Efficient training:** All tokens contribute a supervised next-token prediction, allowing the model to learn from every position simultaneously.
- **Autoregressive behavior preserved:** Even though computation is parallel, each prediction still follows the correct conditional structure $Pr(t_n \mid t_1, \ldots, t_{n-1})$.

Masked attention

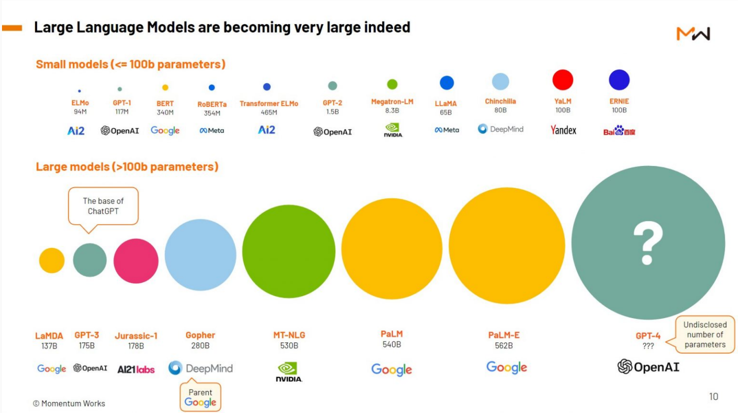


Example: by S. Prince

GPT3

- Sequence lengths are 2048 tokens long
- Batch size is 3.2 million tokens.
- 96 transformer layers, each of which processes a word embedding of size 12288.
- 96 heads in the self-attention layers and the value, query, and key dimension is 128.
- 300 billion tokens
- 175 billion parameters

LLM becoming very large indeed



from: `https://thelowdown.momentum.asia/the-emergence-of-large-language-models-llms/`

Ethical challenges
How to preserve privacy?

Generative models and data leakage

Generative models can potentially memorise and regenerate their training data points.

Extracting Training Data from Diffusion Models

Nicholas Carlini[*1] *Jamie Hayes*[*2] *Milad Nasr*[*1]
Matthew Jagielski[+1] *Vikash Sehwag*[+4] *Florian Tramèr*[+3]
Borja Balle[†2] *Daphne Ippolito*[†1] *Eric Wallace*[†5]
[1]Google [2]DeepMind [3]ETHZ [4]Princeton [5]UC Berkeley
[*]Equal contribution [+]Equal contribution [†]Equal contribution

Generative models and data leakage

Generative models can potentially memorise and regenerate their training data points.

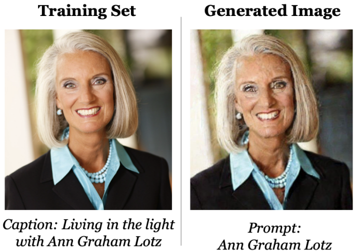


Figure 1: Diffusion models memorize individual training examples and generate them at test time. **Left:** an image from Stable Diffusion's training set (licensed CC BY-SA 3.0, see [49]). **Right:** a Stable Diffusion generation when prompted with "Ann Graham Lotz". The reconstruction is nearly identical (ℓ_2 distance = 0.031).

Data leakage

As we saw in the text generation lab, machine learning algorithms are prone to leak information about their training data:

> arm towards the viewer. Gregor then turned to look out the window at a barren
> sister only needed to hear the visitor's first words of greeting and he knew who
> calm, "I'll get dressed straight away now, pack up my samples and set off. Will
> again, "seven o'clock, and there's still a fog like this." And he lay there sighing,
> harder than before, if that was possible, he felt that the lower part of his body a

Here, our generative model revealed entire sentences from the training input. This is a quality issue, but can also be a privacy issue.

Data leakage

Many modern ML systems trained on user data.

- Smart Compose in Gmail (trained on user emails).
- Generative AI for medical record taking (trained on patient health data).
- Github Copilot trained on public and private repositories.

Even if models do not directly generate private data, it can sometimes be extracted from them.

Data leakage

Training data extraction attacks can reconstruct verbatim training examples e.g., they can extract secrets such as verbatim social security numbers or passwords.

Extracting Training Data from Large Language Models

Nicholas Carlini[1] Florian Tramèr[2] Eric Wallace[3] Matthew Jagielski[4]
Ariel Herbert-Voss[5,6] Katherine Lee[1] Adam Roberts[1] Tom Brown[5]
Dawn Song[3] Úlfar Erlingsson[7] Alina Oprea[4] Colin Raffel[1]
[1]*Google* [2]*Stanford* [3]*UC Berkeley* [4]*Northeastern University* [5]*OpenAI* [6]*Harvard* [7]*Apple*

Abstract

It has become common to publish large (billion parameter) language models that have been trained on private datasets. This paper demonstrates that in such settings, an adversary can perform a *training data extraction attack* to recover individual training examples by querying the language model.

We demonstrate our attack on GPT-2, a language model trained on scrapes of the public Internet, and are able to extract hundreds of verbatim text sequences from the model's training data. These extracted examples include (public) personally identifiable information (names, phone numbers, and email addresses), IRC conversations, code, and 128-bit UUIDs. Our attack is possible even though each of the above sequences are included in just *one* document in the training data.

We comprehensively evaluate our extraction attack to understand the factors that contribute to its success. Worryingly, we find that larger models are more vulnerable than smaller models. We conclude by drawing lessons and discussing possible safeguards for training large language models.

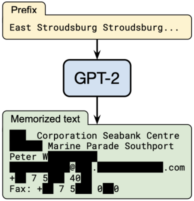


Figure 1: **Our extraction attack.** Given query access to a neural network language model, we extract an individual person's name, email address, phone number, fax number, and physical address. The example in this figure shows information that is all accurate so we redact it to protect privacy.

1 Introduction

The privacy challenge

How do we balance privacy concerns with the desire to train models on as much data as possible?

Formalizing privacy

There have been many many attempts to formalize what it means for a machine learning algorithm or system to be private.

Calibrating Noise to Sensitivity in Private Data Analysis

Cynthia Dwork[1], Frank McSherry[1], Kobbi Nissim[2], and Adam Smith[3*]

[1] Microsoft Research, Silicon Valley. {dwork,mcsherry}@microsoft.com
[2] Ben-Gurion University. kobbi@cs.bgu.ac.il
[3] Weizmann Institute of Science. adam.smith@weizmann.ac.il

Differential Privacy has become the gold standard definition.

Clear theoretical founding, widely used in implemented systems (TensorFlow, US Census statistics, Apple User data, etc.)

Differential privacy

Definition based on notation of neighboring datasets.

Definition: A dataset $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_n]$ is neighbors of a dataset $\mathbf{X}' = [\mathbf{x}'_1, \ldots, \mathbf{x}'_n]$ if:

$$\mathbf{x}_i = \mathbf{x}'_i \text{ for all but one value of } i \in \{1, \ldots n\}.$$

I.e., $\mathbf{x}_j \neq \mathbf{x}'_j$ for a single index j.

Alternative but closely related definition: $\mathbf{X}$ and $\mathbf{X}'$ are neighbors if $\mathbf{X}'$ can be obtained by adding or removing a single data point from $\mathbf{X}$.

Differential privacy

Definition

An algorithm $\mathcal{A}$ satisfies ϵ-differential privacy if, for any two neighboring datasets $\mathbf{X}$, $\mathbf{X}'$, and any possible output of the algorithm $\mathbf{z}$,

$$\Pr[\mathcal{A}(\mathbf{X}) = \mathbf{z}] \le e^{\epsilon} \Pr[\mathcal{A}(\mathbf{X}') = \mathbf{z}].$$

In the context of machine learning, $\mathcal{A}$ could be the training procedure and $\mathbf{z}$ could be, e.g., the model weights.

In the context of databases/statistical applications, $\mathcal{A}$ might implement a simple statistic function like the mean:

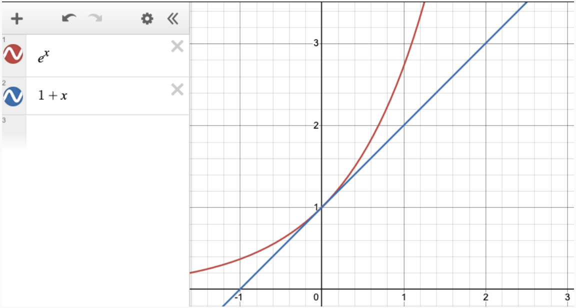
$$\frac{1}{n}\sum_{i=1}^{n} x_i.$$

Differential privacy

Definition

An algorithm $\mathcal{A}$ satisfies ϵ-differential privacy if, for any two neighboring datasets $\mathbf{X}$, $\mathbf{X}'$, and any possible output of the algorithm $\mathbf{z}$, $\Pr[\mathcal{A}(\mathbf{X}) = \mathbf{z}] \leq e^{\epsilon} \Pr[\mathcal{A}(\mathbf{X}') = \mathbf{z}]$.

Think of ϵ as a reasonably small constant. E.g. $\epsilon \in (0, 5]$. For small ϵ, $e^{\epsilon} \approx (1 + \epsilon)$.



Differential privacy

Definition

An algorithm $\mathcal{A}$ satisfies ϵ-differential privacy if, for any two neighboring datasets $\mathbf{X}$, $\mathbf{X}'$, and any possible output of the algorithm $\mathbf{z}$, $\Pr[\mathcal{A}(\mathbf{X}) = \mathbf{z}] \leq e^{\epsilon} \Pr[\mathcal{A}(\mathbf{X}') = \mathbf{z}]$.

In words, differential privacy says that including an individuals data in a dataset $\mathbf{X}$ can only increase or decrease the probability of observing any particular output by a small factor.

Inherently a property of randomized algorithms. Obtaining differentially private machine learning methods will require **adding randomness to the training process**.

Differential privacy properties

Postprocessing property: If an algorithm $\mathcal{A}(\mathbf{X})$ is ϵ-DP, then $\mathcal{B}(\mathcal{A}(\mathbf{X}))$ is ϵ-DP for any (possibly non-private) algorithm $\mathcal{B}$.

Composition property: If an algorithm $\mathcal{A}_1$ is ϵ_1-DP and $\mathcal{A}_2$ is ϵ_2-DP, then $\mathcal{B}(\mathcal{A}_1(\mathbf{X}), \mathcal{A}_2(\mathbf{X}))$ is $(\epsilon_1 + \epsilon_2)$-DP.

There are many ways to add randomness. Perhaps the most common is noise injection.

Simple example: Suppose $\mathbf{X}$ contains scalar values $x_1, \ldots, x_n \in \{0, 1\}$. Suppose we want to compute the average, $Q(\mathbf{X}) = \frac{1}{n}\sum_{i=1}^{n} x_i$.

Naively, adding or removing a point from the dataset changes the average by $\pm\frac{1}{n}$ with probability 1, so, naively, a mean computation is not differentially private.

Noise injection

Differentially Private Estimate of $Q(\mathbf{X}) = \frac{1}{n}\sum_{i=1}^{n} x_i$:

- Generate an appropriate random number η.
- Return $Q(\mathbf{X}) + \eta$.

Example $= \mathbf{X} = \{0, 1, 1, 0, 0, 0\}, \mathbf{X}' = \{0, 1, 1, 0, 1, 0\}$.

Trade-off between <u>privacy</u> and <u>accuracy</u>.

What type of noise and how much?

Theorem (Laplace Mechanism)

For a function Q with <u>*sensitivity*</u> Δ_Q*,*

$$\mathcal{A}(\mathbf{X}) = Q(\mathbf{X}) + Lap(\Delta_Q/\epsilon)$$

is ϵ-differentially private.

Sensitiviy $\Delta_Q = \max_{\text{neighboring } \mathbf{X}, \mathbf{X}'} |Q(\mathbf{X}) - Q(\mathbf{X}')|$.

What is Δ_Q for $Q(\mathbf{X}) = \frac{1}{n}\sum_{i=1}^{n} x_i$?

$Lap(b)$ is a Laplacian random variable with parameter b (which means variance $2b^2$). PDF is:

$$p_b(\eta) = \frac{1}{2b} e^{-|\eta|/b}$$

Laplace mechanism analysis

Theorem (Laplace Mechanism)

For a function Q with sensitivity Δ_Q,
$\mathcal{A}(\mathbf{X}) = Q(\mathbf{X}) + Lap(\Delta_Q/\epsilon)$ *is* ϵ*-differentially private.*

Proof: For any possible output z,

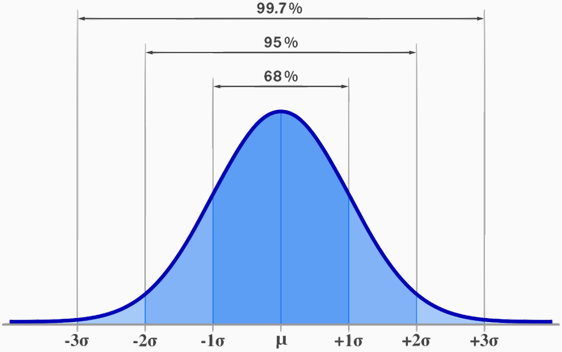
- $\Pr[\mathcal{A}(\mathbf{X}) = z] = \frac{1}{2(\Delta_Q/\epsilon)} e^{-|Q(\mathbf{X})-z|/(\Delta_Q/\epsilon)}$
- $\Pr[\mathcal{A}(\mathbf{X}') = z] = \frac{1}{2(\Delta_Q/\epsilon)} e^{-|Q(\mathbf{X}')-z|/(\Delta_Q/\epsilon)}$

$$\frac{\Pr[\mathcal{A}(\mathbf{X}) = z]}{\Pr[\mathcal{A}(\mathbf{X}') = z]} = e^{-(|Q(\mathbf{X})-z|-|Q(\mathbf{X}')-z|)/(\Delta_Q/\epsilon)}$$

$$\leq e^{\frac{|Q(\mathbf{X})-Q(\mathbf{X}')|}{\Delta_Q/\epsilon}} \leq e^{\epsilon}.$$

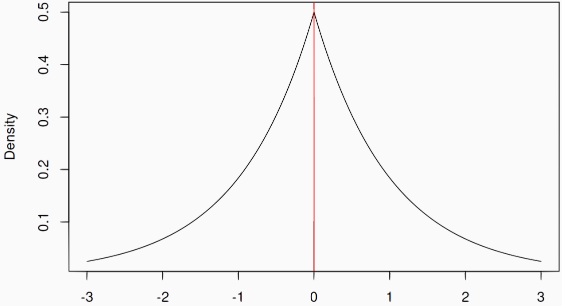
What do we pay in terms of accuracy?

$Lap(b)$ has standard deviation $\sqrt{2}b$. Like Gaussian distribution, Laplace random variables usually fall within a few standard deviations of the mean:



What do we pay in terms of accuracy?

$Lap(b)$ has standard deviation $\sqrt{2}b$. Like Gaussian distribution, Laplace random variables usually fall within a few standard deviations of the mean:



What do we pay in terms of accuracy?

Standard deviation $= \sqrt{2} \cdot \frac{\Delta_Q}{\epsilon}$.

For $x_1, \ldots, x_n \in [0, 1]$, $Q(\mathbf{X}) = \frac{1}{n} \sum_{i=1}^{n} x_i$, we have that:

$$\Delta_Q = \frac{1}{n}.$$

Overall error from adding noise:

$$O\left(\frac{1}{\epsilon n}\right)$$

Very reasonable if n is large!

E.g., if $n = 10,000$ can get error roughly .001 on mean estimate with privacy parameter $\epsilon = .1$.

What about more complex functions?

In machine learning applications, Q is an entire training procedure, and the output is vector of parameters.

$$Q(\mathbf{X}, \mathbf{y}) \rightarrow \boldsymbol{\beta} \in \mathbb{R}^d.$$

Challenges:

- Very hard to estimate the sensitivity to figure out how much noise should be added.
- If some parameters are more sensitive to noise, we could change the models output drastically.

Differentially private (stochastic) gradient descent

Main idea: Typically $Q(\mathbf{X}, \mathbf{y})$ is computed by running gradient descent on a loss function $L(\beta)$. Instead of directly adding noise to $Q(\mathbf{X}, \mathbf{y})$, add noise at each step of gradient descent.

Basic Gradient descent algorithm:

- Choose starting point $\beta^{(0)}$.
- For $i = 0, \ldots, T$:
    - $\beta^{(i+1)} = \beta^{(i)} - \eta \nabla L(\beta^{(i)})$
- Return $\beta^{(T)}$.

Differentially private (stochastic) gradient descent

Typical loss function in machine learning have finite sum structure.

$$L(\boldsymbol{\beta}) = \sum_{j=1}^{n} \ell(\boldsymbol{\beta}, \mathbf{x}_j, y_j)$$

By linearity:

$$\nabla L(\boldsymbol{\beta}) = \sum_{j=1}^{n} \nabla \ell(\boldsymbol{\beta}, \mathbf{x}_j, y_j)$$

Looks just like our mean estimation problem! Can bound the contribution of each data example $(\mathbf{x}_j, y_j)$ to the gradient to get a sensitivity, then add noise.

Differentially private (stochastic) gradient descent

Due to a 2016 paper by Martín Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, Li Zhang.

DP-SGD:

- Choose starting point $\beta^{(0)}$.
- For $i = 0, \ldots, T$:
  - $\beta^{(i+1)} = \beta^{(i)} - \eta(\nabla L(\beta^{(i)}) + \mathbf{r}_i)$
- Return $\beta^{(T)}$.

Above each $\mathbf{r}_i$ is a random Gaussian vector.

Leading way to incorperate privacy into training machine learning models. Implented natively, e.g., in TensorFlow.